

### ■ 1.8.13 Advanced Topic: Building Up Two-dimensional Graphics

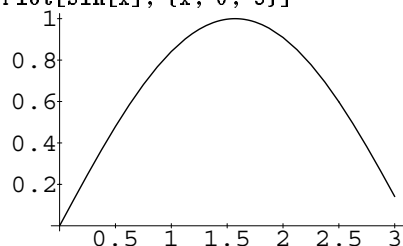
All plots in *Mathematica* are built up from lists of *graphics primitives*. Functions like `Plot` work by producing appropriate lists of graphics primitives, and then displaying them. This section discusses how you can work directly with *Mathematica* graphics primitives. If you only need to make plots of functions or data, then commands like `Plot` and `ListPlot` should be quite sufficient, and you will not need to read this section. If, on the other hand, you want to make more complicated diagrams and pictures in *Mathematica*, then you will need to use the graphics primitives discussed in this section.

|                             |                                            |
|-----------------------------|--------------------------------------------|
| <code>Graphics[list]</code> | representation of two-dimensional graphics |
| <code>Show[graphics]</code> | display graphics                           |

Representation and display of graphics.

`Plot` creates an appropriate graphics object, and then displays it.

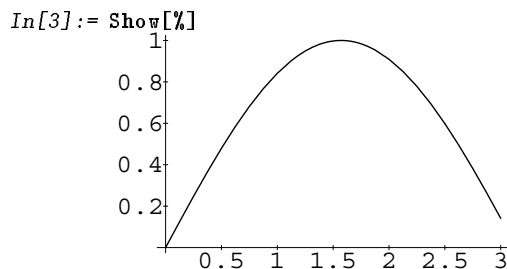
```
In[1]:= Plot[Sin[x], {x, 0, 3}]
```



This shows the internal form of the graphics object. It contains a list of graphics primitives.

```
In[2]:= InputForm[%]
Out[2]//InputForm=
Graphics[{GrayLevel[0],
  {Line[{0., 0.}, {0.125, 0.124675},
    {0.25, 0.247404}, {0.375, 0.366273},
    {0.5, 0.479426}, {0.625, 0.585097},
    {0.75, 0.681639}, {0.875, 0.767544},
    {1., 0.841471}, {1.125, 0.902268},
    {1.1875, 0.927437}, {1.25, 0.948985},
    {1.3125, 0.966827}, {1.375, 0.980893},
    {1.40625, 0.986493},
    {1.4375, 0.991129},
    {1.46875, 0.994798},
    {1.48437, 0.996268}, {1.5, 0.997495},
    {1.51562, 0.998478},
    {1.52344, 0.998879},
    {1.53125, 0.999218},
    {1.53906, 0.999497},
    {1.54687, 0.999714}, {1.55078, 0.9998},
    {1.55469, 0.99987},
    {1.55859, 0.999926},
    {1.5625, 0.999966}, {1.56641, 0.99999},
    {1.57031, 1.}, {1.57422, 0.999994},
    {1.57812, 0.999973},
    {1.58203, 0.999937},
    {1.58594, 0.999885},
    {1.59375, 0.999737},
    {1.60156, 0.999527},
    {1.60937, 0.999256}, {1.625, 0.998531},
    {1.64062, 0.997563},
    {1.65625, 0.996351},
    {1.6875, 0.993198},
    {1.71875, 0.989075}, {1.75, 0.983986},
    {1.8125, 0.970932}, {1.875, 0.954086},
    {2., 0.909297}, {2.125, 0.85032},
    {2.25, 0.778073}, {2.375, 0.693685},
    {2.5, 0.598472}, {2.625, 0.49392},
    {2.75, 0.381661}, {2.875, 0.263446},
    {3., 0.14112}}]}],
{PlotRange -> Automatic,
 AspectRatio -> GoldenRatio^(-1),
 DisplayFunction -> Display["stdout", #1] & ,
 PlotColor -> Automatic, Axes -> Automatic,
 PlotLabel -> None, AxesLabel -> None,
 Ticks -> Automatic, Framed -> False}]
```

Show displays graphics.



|                                                                                             |                                                                                                                                  |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>Point[{x, y}]</code>                                                                  | point at position $x, y$                                                                                                         |
| <code>Line[{{x<sub>1</sub>, y<sub>1</sub>}, {x<sub>2</sub>, y<sub>2</sub>}, ...}]</code>    | line through the points $(x_1, y_1), (x_2, y_2), \dots$                                                                          |
| <code>Rectangle[{xmin, ymin}, {xmax, ymax}]</code>                                          | filled rectangle                                                                                                                 |
| <code>Polygon[{{x<sub>1</sub>, y<sub>1</sub>}, {x<sub>2</sub>, y<sub>2</sub>}, ...}]</code> | filled polygon with the specified list of corners                                                                                |
| <code>CellArray[{{a<sub>11</sub>, a<sub>12</sub>, ...}, {a<sub>21</sub>, ...}, ...}]</code> | rectangular array of gray levels between 0 and 1                                                                                 |
| <code>CellArray[array, {{xmin, ymin}, {xmax, ymax}}, {zmin, zmax}]</code>                   | array of gray levels between $zmin$ and $zmax$ drawn in the rectangle defined by $xmin, ymin, xmax, ymax$                        |
| <code>Text[expr, {x, y}]</code>                                                             | the text of <i>expr</i> , centered at the point $(x, y)$ (see below)                                                             |
| <code>GrayLevel[i]</code>                                                                   | display objects that follow at an intensity (gray level) $i$ between 0 ( <i>black</i> ) and 1 ( <i>white</i> )                   |
| <code>RGBColor[r, g, b]</code>                                                              | display (if possible) with the specified red, green and blue color intensities (each between 0 and 1) (see below)                |
| <code>PointSize[s]</code>                                                                   | give points a size $s$ measured as a fraction of the width of whole plot                                                         |
| <code>Thickness[t]</code>                                                                   | display lines with a thickness $t$ measured as a fraction of the width of whole plot                                             |
| <code>Dashing[{d<sub>1</sub>, d<sub>2</sub>, ...}]</code>                                   | display lines as a sequence of dashed segments, with lengths $d_1, d_2, \dots$ measured as a fraction of the width of whole plot |

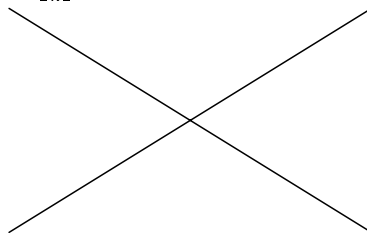
## Two-dimensional graphics primitives.

Here are graphics primitives representing two lines.

```
In[4]:= g = Graphics[ {Line[{{-1,-1},{1,1}}],
                      Line[{{-1,1},{1,-1}}]} ]
Out[4]= -Graphics-
```

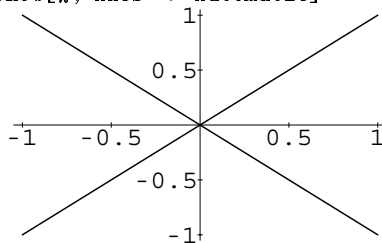
Show displays the graphics.

```
In[5] := Show[%]
```



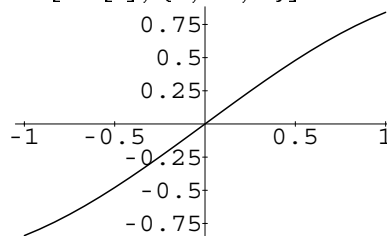
You can use any of the standard options for Show. When you create your own graphics, the default setting for Axes is None.

```
In[6] := Show[%, Axes -> Automatic]
```



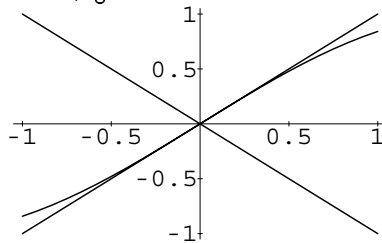
Here is an ordinary Mathematica plot.

```
In[7] := Plot[Sin[x], {x, -1, 1}]
```



You can combine this with your graphics primitives.

```
In[8] := Show[%, g]
```

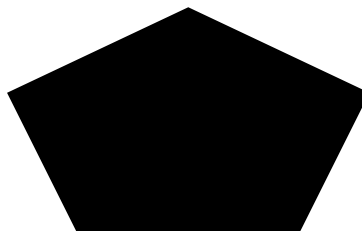


Here are the coordinates of the corners of a pentagon.

```
In[9] := pentagon =  
Table[ N[{Sin[2 Pi n/5], Cos[2 Pi n/5]}], {n, 5} ]  
Out[9] = {{0.951057, 0.309017}, {0.587785, -0.809017},  
{-0.587785, -0.809017}, {-0.951057, 0.309017},  
{0., 1.}}
```

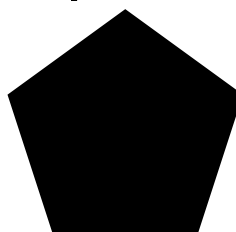
This displays the pentagon. With the default choice of aspect ratio, it comes out somewhat squashed.

```
In[10]:= Show[ Graphics[ Polygon[pentagon] ] ]
```



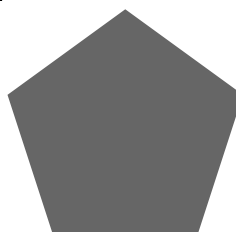
This chooses the aspect ratio so that the shape of the pentagon is preserved.

```
In[11]:= Show[ %, AspectRatio -> Automatic ]
```



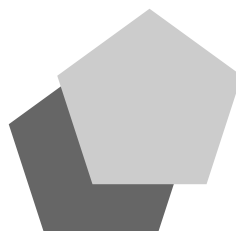
This gives a pentagon with intensity 0.4. Remember that intensity 0 is black, and 1 is white.

```
In[12]:= Show[ Graphics[ { GrayLevel[0.4],  
Polygon[pentagon] } ],  
AspectRatio -> Automatic ]
```



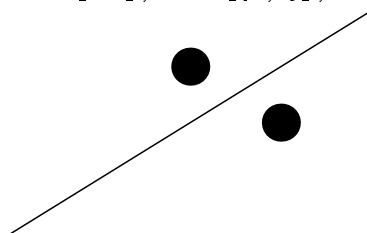
This draws two pentagons superimposed. Later graphics primitives always overwrite earlier ones.

```
In[13]:= Show[ Graphics[  
{ GrayLevel[0.4], Polygon[pentagon],  
GrayLevel[0.8],  
Polygon[pentagon + Table[{0.5, 0.5}, {5}]]  
} ], AspectRatio -> Automatic ]
```



Remember that the scales given in the graphics primitives `PointSize`, `Thickness` and `Dashing` are always measured relative to the width of the whole plot.

```
In[14]:= Show[ Graphics[ { Line[{{-2, -2}, {2, 2}}],
                          PointSize[0.1], Point[{0,1}], Point[{1,0}] } ] ]
```



| <i>option name</i>       | <i>default value</i>       |                                                                                     |
|--------------------------|----------------------------|-------------------------------------------------------------------------------------|
| <code>PlotRange</code>   | <code>Automatic</code>     | range of coordinates to include in the plot ( <code>All</code> includes everything) |
| <code>AspectRatio</code> | <code>1/GoldenRatio</code> | <code>Automatic</code> preserves the shapes of objects                              |
| <code>Axes</code>        | <code>None</code>          | whether axes are drawn                                                              |
| <code>PlotColor</code>   | <code>True</code>          | whether color is used                                                               |

#### Some options for `Graphics` and `Show`.

There are two ways to specify the positions of objects in a plot: absolute coordinates and scaled coordinates.

|                             |                                                           |
|-----------------------------|-----------------------------------------------------------|
| <code>{x, y}</code>         | absolute coordinate values                                |
| <code>Scaled[{x, y}]</code> | scaled coordinates, running from 0 to 1 in each direction |

#### Two ways to specify position in a plot.

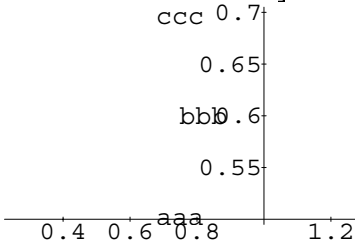
You can use the graphics primitive `Text` to include text in your plots.

|                                           |                                                                                                       |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>Text[expr, {x, y}]</code>           | text centered at $(x, y)$                                                                             |
| <code>Text[expr, {x, y}, {-1, 0}]</code>  | left-hand end at $(x, y)$                                                                             |
| <code>Text[expr, {x, y}, {1, 0}]</code>   | right-hand end at $(x, y)$                                                                            |
| <code>Text[expr, {x, y}, {0, 1}]</code>   | centered above $(x, y)$                                                                               |
| <code>Text[expr, {x, y}, {0, -1}]</code>  | centered below $(x, y)$                                                                               |
| <code>Text[expr, {x, y}, {dx, dy}]</code> | positioned so that $(x, y)$ is at relative coordinates $(dx, dy)$ within the box that bounds the text |

#### Ways to specify the position of text in a plot.

Here is text at various positions in a plot.

```
In[15]:= Show[ Graphics[ { Text[aaa, {0.75, 0.5}],
Text[bbb, {0.75, 0.6}, {-1, 0}],
Text[ccc, {0.75, 0.7}, {0, 1}] } ],
  Axes -> Automatic ]
```



Exactly what font and style are used for the text in your plots will depend on the particular device and particular POSTSCRIPT interpreter you have. Section 2.6.10 discusses how you can get some control over the form of text in graphics.

If you have a color display device, you can use the `RGBColor` graphics primitive to generate color output. The external file `Colors.m` contains many specifications for colors.

|                   |                                                      |
|-------------------|------------------------------------------------------|
| Black             | GrayLevel[0]                                         |
| White             | GrayLevel[1]                                         |
| Gray              | GrayLevel[0.5]                                       |
| Red               | RGBColor[1,0,0]                                      |
| Green             | RGBColor[0,1,0]                                      |
| Blue              | RGBColor[0,0,1]                                      |
| Yellow            | RGBColor[1,1,0]                                      |
| Cyan              | RGBColor[0,1,1]                                      |
| Magenta           | RGBColor[1,0,1]                                      |
| HSBColor[h, s, b] | color specified using hue, saturation and brightness |

Some color specifications from the file Colors.m.

You can use graphics primitives to specify different *styles* for curves drawn by Plot and ListPlot.

|                                                                  |                                                                 |
|------------------------------------------------------------------|-----------------------------------------------------------------|
| PlotStyle -> style                                               | specify a style for all curves                                  |
| PlotStyle -> {{style <sub>1</sub> }, {style <sub>2</sub> }, ...} | specify styles to be used (cyclically) for a sequence of curves |
| GrayLevel[0.5]                                                   | gray                                                            |
| RGBColor[1,0,0], etc.                                            | red, etc.                                                       |
| Thickness[0.05]                                                  | thick                                                           |
| Dashing[{0.05, 0.05}]                                            | dashed                                                          |
| Dashing[{0.01, 0.05, 0.05, 0.05}]                                | dot-dashed                                                      |

Styles for curves.

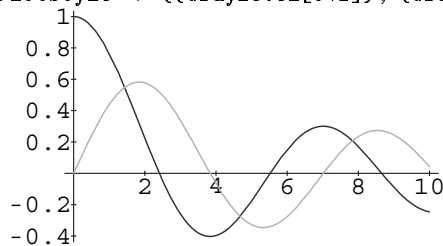
Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: [info@wolfram.com](mailto:info@wolfram.com); <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.



This makes a plot with the first curve having intensity 0.2, and the second one having intensity 0.7. Remember that intensity 0 is black. Notice that the graphics primitives for each curve have to be enclosed in a separate list.

```
In[16]:= Plot[{BesselJ[0, x], BesselJ[1, x]}, {x, 0, 10},
  PlotStyle -> {{GrayLevel[0.2]}, {GrayLevel[0.7]}}]
```



This makes a dashed curve.

```
In[17]:= Plot[ArcTan[x], {x, 0, 10},
  PlotStyle -> Dashing[{0.05, 0.05}]]
```

